

Java String Interview Guide

Simple Language • Real Java Examples • Output Questions • Easy Diagrams • Senior SDET Follow-ups

This version is deliberately different from a textbook: every concept is explained in simple words, followed by a small Java example and an easy mental diagram. Use the code first, predict the output, then read the explanation.

The Most Important Mental Model

```
String s = "apple";  
s = "mango";
```

Before reassignment	After reassignment
s ■■■■■→ "apple"	s ■■■■■→ "mango"
String object is unchanged	"apple" was not edited

Remember: reassignment is not mutation. This single distinction answers many String interview questions.

Contents

1. String Basics & Immutability — 6 questions
2. String Pool — Easy Diagrams — 5 questions
3. == vs equals() — Most Important Interview Area — 5 questions
4. Concatenation & Reassignment — 4 questions
5. StringBuilder & StringBuffer — 5 questions
6. Common String API — With Examples — 6 questions
7. String Coding Questions — Example Solutions — 7 questions
8. Tricky Output Questions — 6 questions
9. Senior SDET / Framework Questions — 6 questions
10. Rapid-Fire Revision — Example Included — 8 questions

1. 1. String Basics & Immutability

Q1. What happens when we reassign a String?

■ I can explain ■ I can predict output ■ I can code it

```
String s = "apple";  
s = "mango";  
System.out.println(s);
```

Answer: Output: mango

Simple explanation: s is a reference. First it points to "apple". Then we make it point to "mango". We did not change "apple".

Diagram: s → "apple" then s → "mango".

Q2. Why is String immutable?

■ I can explain ■ I can predict output ■ I can code it

```
String s = "Java";  
s.concat(" Selenium");  
System.out.println(s);
```

Answer: Output: Java

Simple explanation: concat() creates/returns another String. Because we did not store the returned value, s still points to "Java".

Q3. Prove String is immutable.

■ I can explain ■ I can predict output ■ I can code it

```
String s = "Java";  
String old = s;  
s = s.concat(" Test");  
System.out.println(old);  
System.out.println(s);
```

Answer: Output: Java / Java Test

Simple explanation: the old object was not changed. A new value was returned and assigned to s.

Q4. Is String a primitive?

■ I can explain ■ I can predict output ■ I can code it

```
String name = "Swapnil";
```

Answer: Answer: No. String is a class/object type. The variable stores a reference to a String object.

Q5. Can a String object be modified?

■ I can explain ■ I can predict output ■ I can code it

```
String s = "hello";  
s.toUpperCase();  
System.out.println(s);
```

Answer: Output: hello

Why? toUpperCase() returns a new String. It does not modify the original.

Q6. What happens to the old value after reassignment?

■ I can explain ■ I can predict output ■ I can code it

```
String s = "apple";  
s = "mango";
```

Answer: Simple explanation: if no reachable reference uses the old object, it may become eligible for garbage collection. String literals are managed through JVM string-pool mechanisms.

2. 2. String Pool — Easy Diagrams

Q7. What is the String Pool?

■ I can explain ■ I can predict output ■ I can code it

```
String a = "Java";  
String b = "Java";
```

Answer: Result: both references can point to the same pooled String.

Diagram: a ■■

b ■■■■ → "Java" (String Pool)

Why? Java reuses identical string literals.

Q8. Predict the output: literal vs literal.

■ I can explain ■ I can predict output ■ I can code it

```
String a = "Java";  
String b = "Java";  
System.out.println(a == b);
```

Answer: Output: true

Simple explanation: both references point to the same pooled literal.

Q9. Predict the output: new String vs new String.

■ I can explain ■ I can predict output ■ I can code it

```
String a = new String("Java");  
String b = new String("Java");  
System.out.println(a == b);
```

Answer: Output: false

Diagram: a → Object #1 "Java"

b → Object #2 "Java"

Important: same content, different objects.

Q10. Predict: new String vs literal.

■ I can explain ■ I can predict output ■ I can code it

```
String a = new String("Java");  
String b = "Java";  
System.out.println(a == b);  
System.out.println(a.equals(b));
```

Answer: Output: false, true.

Simple: == checks reference identity; equals() checks String content.

Q11. What does intern() do?

■ I can explain ■ I can predict output ■ I can code it

```
String a = new String("Java");  
String b = a.intern();  
String c = "Java";  
System.out.println(b == c);
```

Answer: Output: true.

Simple: intern() gives the canonical pooled String for that content.

3. 3. == vs equals() — Most Important Interview Area

Q12. What is the difference between == and equals()?

■ I can explain ■ I can predict output ■ I can code it

```
String a = new String("hello");  
String b = new String("hello");
```

Answer: ==: asks "Are these the same object/reference?"

equals(): asks "Do these Strings contain the same characters?"

Q13. Predict output: pooled literals.

■ I can explain ■ I can predict output ■ I can code it

```
String s1 = "hello";  
String s2 = "hello";  
System.out.println(s1 == s2);  
System.out.println(s1.equals(s2));
```

Answer: Output: true / true.

Q14. Predict output: objects.

■ I can explain ■ I can predict output ■ I can code it

```
String s1 = new String("hello");  
String s2 = new String("hello");  
System.out.println(s1 == s2);  
System.out.println(s1.equals(s2));
```

Answer: Output: false / true.

Q15. Why does == sometimes appear to work for Strings?

■ I can explain ■ I can predict output ■ I can code it

```
String a = "QA";  
String b = "QA";
```

Answer: Because literals are pooled. Do not use this as a reason to compare String content with ==. Use equals().

Q16. How do you safely compare a possibly-null String?

■ I can explain ■ I can predict output ■ I can code it

```
String actual = null;  
System.out.println("PASS".equals(actual));
```

Answer: Output: false.

Why? The constant "PASS" is non-null, so calling equals on it is safe.

4. 4. Concatenation & Reassignment

Q17. What happens with +?

■ I can explain ■ I can predict output ■ I can code it

```
String s = "Java";  
s = s + " Selenium";  
System.out.println(s);
```

Answer: Output: Java Selenium.

Simple: the original String is not edited. A result is produced and assigned back to s.

Q18. What is the danger of repeated + in a loop?

■ I can explain ■ I can predict output ■ I can code it

```
String result = "";  
for (int i = 0; i < 1000; i++) {  
    result = result + i;  
}
```

Answer: Simple: repeated concatenation can create many intermediate String results. For repeated building, use StringBuilder.

Q19. Why does compile-time concatenation matter?

■ I can explain ■ I can predict output ■ I can code it

```
String a = "Ja" + "va";  
String b = "Java";  
System.out.println(a == b);
```

Answer: Output: true for this compile-time constant expression. The compiler can fold it to the same constant value.

Q20. Predict: variable concatenation.

■ I can explain ■ I can predict output ■ I can code it

```
String x = "Ja";  
String a = x + "va";  
String b = "Java";  
System.out.println(a == b);
```

Answer: Typical output: false. The concatenation involves a variable and is evaluated at runtime rather than being the same compile-time constant expression.

5. 5. StringBuilder & StringBuffer

Q21. String vs StringBuilder?

■ I can explain ■ I can predict output ■ I can code it

```
String s = "A";
StringBuilder sb = new StringBuilder("A");
sb.append("B");
```

Answer: String: immutable.

StringBuilder: mutable; append changes the builder's current contents.

Diagram: String → new value; Builder → same mutable object grows.

Q22. Predict output after concat vs append.

■ I can explain ■ I can predict output ■ I can code it

```
String s = "A";
s.concat("B");
System.out.println(s);

StringBuilder sb = new StringBuilder("A");
sb.append("B");
System.out.println(sb);
```

Answer: Output: A / AB.

Reason: String concat returns a new value; StringBuilder append modifies the builder.

Q23. StringBuilder vs StringBuffer?

■ I can explain ■ I can predict output ■ I can code it

```
StringBuilder sb = new StringBuilder();
StringBuffer sf = new StringBuffer();
```

Answer: Simple: both are mutable. StringBuffer synchronizes its methods; StringBuilder does not. StringBuilder is generally preferred for ordinary single-threaded building.

Q24. What is capacity?

■ I can explain ■ I can predict output ■ I can code it

```
StringBuilder sb = new StringBuilder();
System.out.println(sb.length());
System.out.println(sb.capacity());
```

Answer: Simple: length = characters currently stored. capacity = buffer space available before it needs to grow. Exact initial/growth values can be implementation/API-version dependent.

Q25. How do you reverse a String?

■ I can explain ■ I can predict output ■ I can code it

```
String input = "Java";
String output = new StringBuilder(input).reverse().toString();
System.out.println(output);
```

Answer: Output: avaJ.

6. 6. Common String API — With Examples

Q26. substring()

■ I can explain ■ I can predict output ■ I can code it

```
String s = "Automation";  
System.out.println(s.substring(0, 4));
```

Answer: Output: Auto.

Remember: start index is inclusive; end index is exclusive.

Q27. charAt()

■ I can explain ■ I can predict output ■ I can code it

```
String s = "Java";  
System.out.println(s.charAt(1));
```

Answer: Output: a. Index starts at 0.

Q28. indexOf()

■ I can explain ■ I can predict output ■ I can code it

```
String s = "selenium";  
System.out.println(s.indexOf("e"));
```

Answer: Output: 1 — the first matching position.

Q29. replace() vs replaceAll()

■ I can explain ■ I can predict output ■ I can code it

```
String s = "a1b2";  
System.out.println(s.replace("1", "X"));  
System.out.println(s.replaceAll("\\d", "X"));
```

Answer: Output: aXb2 / aXbX.

Simple: replace works with literal character/sequence semantics; replaceAll uses regex.

Q30. trim() vs strip()

■ I can explain ■ I can predict output ■ I can code it

```
String s = " Java ";  
System.out.println(s.trim());  
System.out.println(s.strip());
```

Answer: Both remove surrounding whitespace in this simple example. strip() uses Unicode-aware whitespace semantics.

Q31. isEmpty() vs.isBlank()

■ I can explain ■ I can predict output ■ I can code it

```
System.out.println("").isEmpty();  
System.out.println(" ").isEmpty();  
System.out.println(" ").isBlank();
```

Answer: Output: true / false / true.

Simple: empty means zero characters; blank also treats whitespace-only text as blank.

7. 7. String Coding Questions — Example Solutions

Q32. Reverse a String.

■ I can explain ■ I can predict output ■ I can code it

```
public static String reverse(String s) {  
    return new StringBuilder(s).reverse().toString();  
}
```

Answer: Example: reverse("Java") → "avaJ".

Interview follow-up: Can you do it without StringBuilder? Yes, use two pointers/char array.

Q33. Palindrome check.

■ I can explain ■ I can predict output ■ I can code it

```
public static boolean isPalindrome(String s) {  
    int left = 0, right = s.length() - 1;  
    while (left < right) {  
        if (s.charAt(left++) != s.charAt(right--)) return false;  
    }  
    return true;  
}
```

Answer: Example: "madam" → true; "java" → false.

Simple: compare first/last, then move inward.

Q34. Anagram check.

■ I can explain ■ I can predict output ■ I can code it

```
String a = "listen";  
String b = "silent";  
// Count each character in a map and decrement using b.
```

Answer: Example: listen + silent → true.

Senior point: clarify whether spaces, case, and Unicode characters should be ignored.

Q35. First non-repeating character.

■ I can explain ■ I can predict output ■ I can code it

```
String s = "swiss";  
// First pass: count characters.  
// Second pass: return first character with count == 1.
```

Answer: Example: "swiss" → w.

Why two passes? The first pass establishes complete frequencies; the second preserves original order.

Q36. Remove duplicate characters preserving order.

■ I can explain ■ I can predict output ■ I can code it

```
String s = "programming";  
Set<Character> seen = new LinkedHashSet<>();  
for (char c : s.toCharArray()) {  
    seen.add(c);  
}  
// Build the final String from seen.
```

Answer: Example: programming → progamin (preserving first occurrence order).

Q37. Check String rotation.

■ I can explain ■ I can predict output ■ I can code it

```
String a = "abcd";  
String b = "cdab";  
boolean result = a.length() == b.length() && (a + a).contains(b);
```

Answer: Result: true.

Simple: if b is a rotation of a, it will appear inside a+a.

Q38. Longest substrings without repeating characters.

■ I can explain ■ I can predict output ■ I can code it

```
String s = "abcabcbb";  
// Use a sliding window and a Set/last-seen index map.
```

Answer: Example: "abcabcbb" → "abc" with length 3.

Senior point: aim for O(n), not repeated nested scans.

8. 8. Tricky Output Questions

Q39. Predict this.

■ I can explain ■ I can predict output ■ I can code it

```
String s = "hello";  
s.concat("world");  
System.out.println(s);
```

Answer: Output: hello.

Trap: the returned String was ignored.

Q40. Predict this.

■ I can explain ■ I can predict output ■ I can code it

```
String s = "hello";  
s = s.concat("world");  
System.out.println(s);
```

Answer: Output: helloworld.

Reason: returned value was assigned back to s.

Q41. Predict this.

■ I can explain ■ I can predict output ■ I can code it

```
String s = "Java";  
System.out.println(s.replace("J", "j"));  
System.out.println(s);
```

Answer: Output: java / Java.

Trap: replace does not mutate the original String.

Q42. Predict this.

■ I can explain ■ I can predict output ■ I can code it

```
String s = "Java";  
String t = s;  
s = "Python";  
System.out.println(t);  
System.out.println(s);
```

Answer: Output: Java / Python.

Diagram: initially s and t → "Java". After reassignment, s → "Python", while t still → "Java".

Q43. Predict this.

■ I can explain ■ I can predict output ■ I can code it

```
String a = new String("test");  
String b = a.intern();  
String c = "test";  
System.out.println(a == b);  
System.out.println(b == c);
```

Answer: Output: false / true.

Reason: a is a separately created object; b and c refer to the canonical pooled value.

Q44. Predict this.

■ I can explain ■ I can predict output ■ I can code it

```
String s = null;  
System.out.println("Java".equals(s));
```

Answer: Output: false. This is a safe null comparison pattern.

9. 9. Senior SDET / Framework Questions

Q45. Where do you use String in automation?

■ I can explain ■ I can predict output ■ I can code it

```
String url = baseUrl + "/users";
String token = response.jsonPath().getString("token");
String expected = "ACTIVE";
```

Answer: **Examples:** URLs, tokens, JSON paths, headers, environment names, test data, assertion messages, file paths and report text.

Q46. How can String cause flaky assertions?

■ I can explain ■ I can predict output ■ I can code it

```
String actual = "SUCCESS\n";
String expected = "SUCCESS";
System.out.println(actual.equals(expected));
```

Answer: **Output:** false.

Simple: invisible whitespace/newline matters. Decide whether the API/UI contract requires exact matching, trimming, normalisation, or structured comparison.

Q47. How would you handle null, empty and blank?

■ I can explain ■ I can predict output ■ I can code it

```
String a = null;
String b = "";
String c = " ";
```

Answer: **Simple:** null = no String reference; empty = zero characters; blank = empty or whitespace-only. Do not silently treat them as identical unless the business rule says so.

Q48. Why is String a good HashMap key?

■ I can explain ■ I can predict output ■ I can code it

```
Map<String, Integer> map = new HashMap<>();
map.put("PASS", 1);
```

Answer: **Simple:** String is immutable, so its content/hash value remains stable after insertion. Mutable keys can cause lookup problems if their equality/hash state changes.

Q49. Why not use StringBuilder as a shared mutable field?

■ I can explain ■ I can predict output ■ I can code it

```
class TestData {
    StringBuilder value = new StringBuilder();
}
```

Answer: **Simple:** shared mutable state can create race conditions during parallel tests. Keep test data isolated or use appropriate synchronisation/design.

Q50. Explain String immutability in 30 seconds.

■ I can explain ■ I can predict output ■ I can code it

```
String s = "apple";
s = "mango";
```

Answer: **Interview answer:** "String is immutable, so the existing String object cannot be changed. In this example s first references apple and then is reassigned to mango. The reference changes; the String object does not."

10. 10. Rapid-Fire Revision — Example Included

Q51. Can String be extended?

■ I can explain ■ I can predict output ■ I can code it

```
class MyString extends String { }
```

Answer: **Answer:** No. String is final.

Q52. Can String be used as a Map key?

■ I can explain ■ I can predict output ■ I can code it

```
Map<String, String> users = new HashMap<>();  
users.put("101", "Alice");
```

Answer: **Answer:** Yes. It is a common immutable key type.

Q53. Is StringBuilder mutable?

■ I can explain ■ I can predict output ■ I can code it

```
StringBuilder sb = new StringBuilder("A");  
sb.append("B");
```

Answer: **Answer:** Yes. sb now represents "AB".

Q54. Is StringBuffer mutable?

■ I can explain ■ I can predict output ■ I can code it

```
StringBuffer sb = new StringBuffer("A");  
sb.append("B");
```

Answer: **Answer:** Yes. Its methods are synchronised.

Q55. Does == compare String content?

■ I can explain ■ I can predict output ■ I can code it

```
"A" == new String("A")
```

Answer: **Answer:** No. It compares object references; this expression is false.

Q56. Does equals() compare String content?

■ I can explain ■ I can predict output ■ I can code it

```
"A".equals(new String("A"))
```

Answer: **Answer:** Yes. It is true because the contents match.

Q57. Does concat() modify String?

■ I can explain ■ I can predict output ■ I can code it

```
String s = "A";  
s.concat("B");
```

Answer: **Answer:** No. It returns a new String.

Q58. Does append() modify StringBuilder?

■ I can explain ■ I can predict output ■ I can code it

```
StringBuilder sb = new StringBuilder("A");  
sb.append("B");
```

Answer: **Answer:** Yes. The same mutable builder now contains "AB".

Senior Interview Answer Pattern

- **1. One-line definition:** Answer directly.
- **2. Simple example:** Show 3–6 lines of Java.
- **3. Diagram:** Explain what the reference and object are doing.
- **4. Output:** Say exactly what prints.
- **5. Why:** Explain the internal concept in simple language.
- **6. Framework connection:** Give one real SDET example.

Example senior answer: "String is immutable. In `String s = "apple"; s = "mango";`, we do not change the apple object. We simply make the reference `s` point to mango. That is why the output is mango. In automation, this matters when building test data and assertions because String operations return new values rather than changing the original value."